Mockito Hands-on Exercise Report

Exercise 1: Mocking and Stubbing

Scenario

You need to test a service that depends on an external API. Since the external API may not always be available, Mockito is used to create a mock object and stub its methods to return predefined values.

Objective

The objective of this exercise is to understand how to use Mockito to:

- Create mock objects
- Stub method behavior
- Test service logic independently of external dependencies

---

1. Understanding Mocking and Stubbing

Mocking allows developers to simulate the behavior of real objects.
Stubbing defines what values should be returned when a method is called.

Key concepts:

- mock() → creates a mock object
- when().thenReturn() → defines stub behavior

---

2. Implementation

ExternalApi.java

```
public interface ExternalApi {
   String getData();
}
```

MyService.java

```
public class MyService {

   private ExternalApi api;

   public MyService(ExternalApi api) {
      this.api = api;
```

```
    }

    public String fetchData() {
        return api.getData();
    }
}
```

MyServiceTest.java

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;
import static org.mockito.Mockito.*;

public class MyServiceTest {

    @Test
    public void testExternalApi() {

        ExternalApi mockApi = mock(ExternalApi.class);

        when(mockApi.getData()).thenReturn("Mock Data");

        MyService service = new MyService(mockApi);

        String result = service.fetchData();

        assertEquals("Mock Data", result);
    }
}
```

---

3. Output

- Mock object created successfully

- Stubbed method returned expected value
- Test executed successfully with green status

Expected Output:

Tests passed: 1

---

4. Conclusion

Mockito was successfully used to mock external dependencies and stub method behavior. This ensures isolated and reliable unit testing without relying on external systems.

Exercise 2: Verifying Interactions

Scenario

You need to ensure that a specific method is called on a dependency with correct arguments during execution.

Objective

The objective is to understand interaction testing using Mockito's verify mechanism.

---

1. Understanding Verification

Mockito provides verify() to check whether a method was called on a mock object.

Key concept:

- verify(mock).method() → validates method invocation

---

2. Implementation

MyServiceTest.java

```
import org.junit.Test;

import static org.mockito.Mockito.*;


public class MyServiceTest {


    @Test

    public void testVerifyInteraction() {
```

```
        ExternalApi mockApi = mock(ExternalApi.class);

        MyService service = new MyService(mockApi);

        service.fetchData();

        verify(mockApi).getData();
    }
}
```

---

3. Output

- Method call on mock object verified successfully
- No assertion errors occurred
- Test executed successfully

Expected Output:

Tests passed: 1

---

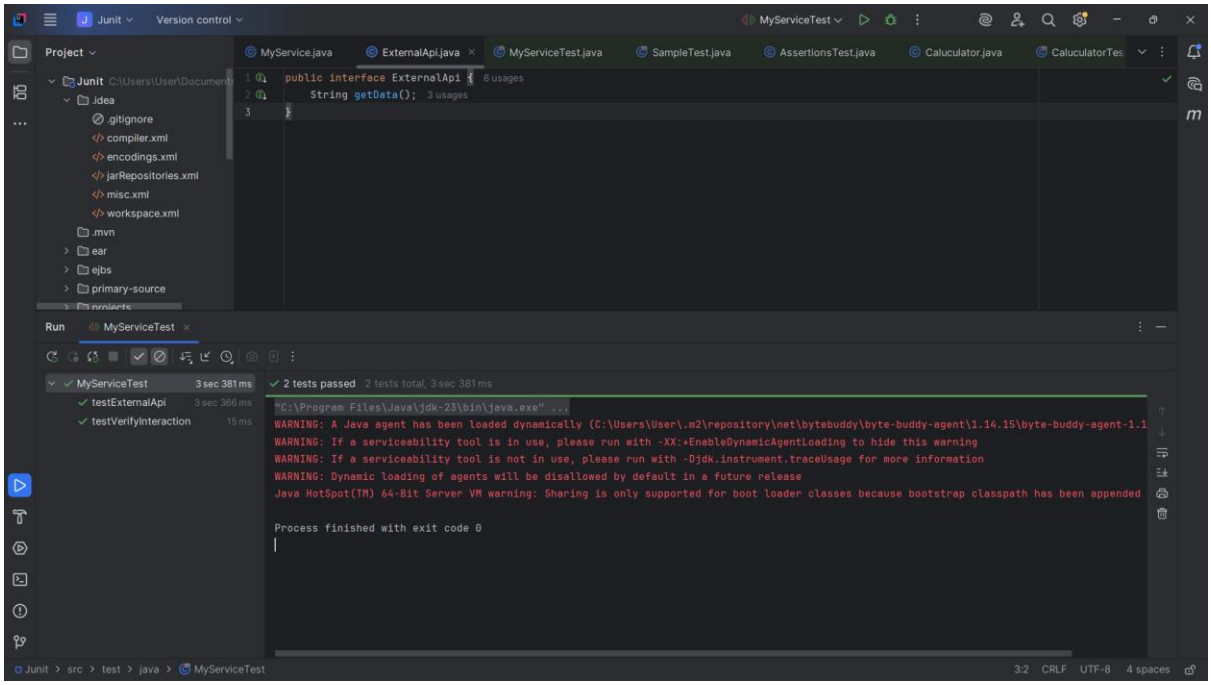
4. Conclusion

Mockito verification ensures that interactions between objects occur as expected. This improves test reliability by validating method calls, not just return values.

Final Conclusion

Both mocking/stubbing and interaction verification were successfully implemented using Mockito. These techniques are essential for writing clean, isolated, and reliable unit tests in Java applications.

OUTPUT SCREENSHOT


Junit
Version control
MyServiceTest
Project
MyService.java
ExternalApi.java
MyServiceTest.java
SampleTest.java
AssertionsTest.java
Calculator.java
public interface ExternalApi { 6 usages
String getData(); 3 usages
}
.idea
.gitignore
compiler.xml
encodings.xml
jarRepositories.xml
misc.xml
workspace.xml
.mvn
ear
ejbs
primary-source
Run
MyServiceTest
MyServiceTest 3 sec 381 ms
testExternalApi 3 sec 366 ms
testVerifyInteraction 15 ms
2 tests passed 2 tests total, 3 sec 381 ms
"C:\Program Files\Java\jdk-23\bin\java.exe" ...
WARNING: A Java agent has been loaded dynamically (C:\Users\User\.m2\repository\net\bytebuddy\byte-buddy-agent\1.14.15\byte-buddy-agent-1.1
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Process finished with exit code 0
Junit › src › test › java › MyServiceTest
3:2 CRLF UTF-8 4 spaces